MEDQUAD HEALTH SOLUTIONS

Web Application — Software Requirements Specification

Assignment 1: Web Engineering

Bahria University • Department of Software Engineering

MERN Stack • AI-Powered • Cloud-Native • HCI-Driven

M ABDULLAH SARWAR -049 | AIZAZ AHMAD -106

Web Engineering | Artificial Intelligence | HCI / UX Design | Cloud Computing

1. Executive Summary & Platform Definition

PLATFORM TYPE	Hybrid SaaS Client/Staff Operations Portal — a cloud-hosted, multi-tenant web application serving three distinct user groups through a single intelligent MERN-stack interface.

MedQuad Health Solutions is a company that imports, installs, and services high-end biomedical equipment including MRI and CT scanners for hospital clients. The proposed web application serves as a unified operational hub: managing service lifecycles, client communication, AI-driven predictive maintenance, equipment inventory, and cloud-native deployment — all within a single MERN-stack codebase.

1.1 User Personas

Persona	Role	Primary Needs
Public Visitor	Procurement officer / Hospital BioMed Engineer	Browse equipment catalog, view specs, initiate business inquiries. No credentials required.
Hospital Client	Hospital Biomedical Dept. Head (Authenticated)	Raise & track service tickets, view equipment health, access manuals, review service history for compliance.

MedQuad Staff	Field Technician + Admin (Two sub-roles)	Technician: mobile-first task queue, parts lookup, ticket updates. Admin: global management dashboard, KPIs, AI predictions.

2. Types of Requirements

As defined in Requirements Engineering, all project requirements fall into two major categories:

Functional Requirements	Non-Functional Requirements
Definition: Describe what the system must do — its capabilities and services. **Sub-categories for MedQuad:** • Data Requirements — how data is stored, retrieved, updated • Interface Requirements — API contracts, UI interactions • Navigational Requirements — routing, access flows • Personalization Requirements — role-based views, preferences • Transactional Requirements — ticket creation, status updates	**Definition:** Describe desired quality levels, performance constraints, and system-wide properties. **Sub-categories for MedQuad:** • Content — correctness and accuracy of displayed data • Quality — performance, reliability, maintainability • System Environment — platform, browser, device support • User Interface — accessibility (WCAG 2.1 AA), responsiveness • Evolution — scalability, extensibility for future modules • Project Constraints — tech stack, timeline, budget

The process of understanding and specifying these requirements produces a formal Requirements Document — which this document constitutes for the MedQuad Health Solutions platform.

3. Functional Requirements

Functional requirements define the specific behaviours and capabilities the MedQuad system must provide. They are organized by the five sub-categories defined in the assignment framework.

TOOL NOTE	Requirements below were identified, categorized, and managed using RMsis for JIRA (requirements management tool). Each requirement carries: Summary, Category, Dependency, Priority, and Criticality attributes as mandated by the assignment.

3.1 Functional Requirements Register

ID	Category	Summary / Description	Dependency	Priority	Crit.
FR-D01	Data	The system shall store complete user profiles including name, email, hashed password, role (public | client | technician | admin), and linked organization ID.	None	High	C1
FR-D02	Data	The system shall maintain an Equipment record for each installed machine, including model, serial number, manufacturer, installation date, and a timestamped usage-hours log array for AI model consumption.	FR-D01	High	C1
FR-D03	Data	The system shall persist Service Tickets with fields: client ID, equipment ID, description, status (open | in-progress | resolved | closed), AI-generated category, AI priority score, assigned technician, and a chronological update log (subdocument array).	FR-D01, FR-D02	High	C1
FR-D04	Data	The system shall store Inventory records for spare parts including part name, part number, compatible equipment models, quantity on hand, and a reorder threshold that triggers low-stock alerts.	None	Medium	C2
FR-D05	Data	The system shall store AI-generated Maintenance Predictions per equipment unit, including predicted failure date, confidence score, model version, and generation timestamp.	FR-D02	High	C1
FR-D06	Data	The system shall maintain a Client (Organization) collection separate from Users, linking multiple user accounts to a single hospital entity.	FR-D01	High	C1
FR-I01	Interface	The system shall expose a RESTful API (v1) following resource-based URL conventions (e.g., GET /api/v1/tickets/:id) for all CRUD operations. All responses shall be in JSON format with standardized error envelopes.	None	High	C1
FR-I02	Interface	The system shall provide a GraphQL endpoint for the Client Dashboard to fetch nested relational data (client → equipment → latest ticket → technician) in a single network round-trip, avoiding REST over-fetching.	FR-I01	Medium	C2
FR-I03	Interface	The system shall integrate Socket.io WebSocket connections so that ticket status changes are pushed in real time to all connected clients in the relevant	FR-D03	High	C1

		Socket.io room without requiring a page refresh.			
FR-I04	Interface	The system shall integrate with AWS S3 via the AWS SDK to generate pre-signed URLs for direct browser-to-S3 upload and download of binary assets (manuals, photos, contracts).	None	Medium	C2
FR-I05	Interface	The system shall integrate with the NLP Microservice via an internal REST call upon ticket creation to retrieve AI-generated category, priority score, and recommended technician specialty.	FR-D03	High	C1
FR-I06	Interface	The system shall integrate with the Predictive Maintenance Microservice via a nightly cron job, passing equipment usage logs and returning predicted failure dates stored in the MaintenancePredictions collection.	FR-D02, FR-D05	High	C1
FR-N01	Navigational	The system shall enforce Role-Based Access Control (RBAC) so that unauthenticated users can only access public routes (/catalog, /about, /contact). All protected routes redirect to /login if the JWT is absent or expired.	FR-D01	High	C1
FR-N02	Navigational	The system shall redirect each authenticated user to their role-specific dashboard upon login: /dashboard/client, /dashboard/technician, or /dashboard/admin.	FR-N01	High	C1
FR-N03	Navigational	The system shall implement a 4-step wizard navigation for ticket creation (Step 1: Select Equipment → Step 2: Describe Problem → Step 3: Review AI Suggestions → Step 4: Confirm & Submit) with the ability to navigate back to any previous step without data loss.	FR-D03	High	C1
FR-N04	Navigational	The system shall provide server-side cursor-based pagination for the Equipment Catalog, accepting query parameters (?search=, &manufacturer=, &page=, &limit=) and returning a consistent dataset regardless of concurrent writes.	FR-D02	Medium	C2
FR-P01	Personalization	The system shall render a distinct dashboard layout per user role. The Client Dashboard shall display a Ticket Status Board and Equipment Health panel. The Technician Dashboard shall display a prioritized task queue. The Admin Dashboard shall display KPI metrics, AI predictions, and global management tables.	FR-D01	High	C1

FR-P02	Personalization	The system shall allow Technicians to set their availability status (available | on-site | off-duty), which the AI auto-assignment engine uses when routing new tickets.	FR-D01	Medium	C2
FR-P03	Personalization	The system shall display all client-facing ticket status labels in plain-language terms (e.g., 'Our team is on their way' instead of 'DISPATCHED') to align with non-technical user mental models.	FR-D03	Medium	C2
FR-T01	Transactional	The system shall allow an authenticated Client user to create a new Service Ticket by completing the 4-step wizard. Upon submission, the system shall: (a) persist the ticket, (b) dispatch to the NLP microservice asynchronously, (c) auto-assign a technician if available, (d) emit a Socket.io event, and (e) send an email notification via SendGrid API.	FR-D03, FR-I05	High	C1
FR-T02	Transactional	The system shall allow a Technician or Admin to update a ticket's status, add update notes, and attach photos. Each update shall be appended to the ticket's update log subdocument with a timestamp and author reference.	FR-D03	High	C1
FR-T03	Transactional	The system shall allow an Admin to create a Preventive Service Ticket pre-populated from a Maintenance Prediction record, with a single-click action from the Admin Dashboard's Predicted Failures widget.	FR-D05	High	C1
FR-T04	Transactional	The system shall allow Admin users to perform full CRUD (Create, Read, Update, Delete) operations on Equipment, Client, and User records through the Admin management interface.	FR-D01, FR-D02, FR-D06	High	C1
FR-T05	Transactional	The system shall trigger a low-stock alert (in-app notification + email) when any Inventory item's quantity on hand falls at or below its reorder threshold during a ticket update that consumes parts.	FR-D04	Medium	C2

4. Non-Functional Requirements

Non-functional requirements define quality attributes, system constraints, and operational properties that the MedQuad platform must satisfy. They cut across all features and are as binding as functional requirements.

ID	Category	Summary / Description	Dependency	Priority	Crit.
NFR-C01	Content	All equipment specifications displayed in the catalog shall be validated against the Equipment collection and shall not display stale data older than 60 seconds (cache TTL enforced via React Query).	FR-D02	High	C1
NFR-C02	Content	AI-generated maintenance predictions shall display the model version and generation timestamp alongside every prediction to ensure content traceability and auditability.	FR-D05	Medium	C2
NFR-Q01	Quality	The REST API shall respond to 95% of requests within 300ms under a simulated load of 200 concurrent users, as measured by Apache JMeter load tests prior to production deployment.	None	High	C1
NFR-Q02	Quality	The system shall achieve a minimum uptime of 99.5% on a rolling 30-day basis, enforced through AWS Multi-AZ ECS deployment with ALB health checks and automatic task replacement.	None	High	C1
NFR-Q03	Quality	All backend business logic (Service Layer functions) shall achieve minimum 80% unit test coverage as enforced by the Jest test suite and reported in the GitHub Actions CI/CD pipeline.	None	Medium	C2
NFR-Q04	Quality	The NLP ticket categorization microservice shall achieve a minimum classification accuracy of 85% on the validation dataset before being approved for production deployment.	FR-I05	High	C1
NFR-S01	Sys. Env.	The frontend React SPA shall be fully functional on the latest two stable versions of Chrome, Firefox, Safari, and Edge browsers on both desktop and mobile devices.	None	High	C1
NFR-S02	Sys. Env.	The Technician Dashboard shall be optimized as a mobile-first Progressive Web App (PWA) with offline ticket-viewing capability via service workers, targeting 375px–768px viewport widths.	FR-P01	Medium	C2
NFR-S03	Sys. Env.	All backend microservices shall be containerized using Docker and deployable on AWS ECS Fargate. No service shall have a dependency on a specific EC2 instance or OS configuration.	None	High	C1

NFR-S04	Sys. Env.	The MongoDB database shall be hosted on MongoDB Atlas (M10 dedicated cluster, AWS us-east-1 region) with automated daily backups and point-in-time recovery enabled.	None	High	C1
NFR-U01	User Interface	The platform shall conform to WCAG 2.1 Level AA accessibility standards, including minimum 4.5:1 colour contrast ratio, keyboard navigability of all interactive elements, and ARIA labels on all icon-only buttons.	None	High	C1
NFR-U02	User Interface	The Client-facing ticket creation wizard shall not require more than 4 steps to complete. Each step shall contain no more than 5 input fields to maintain low cognitive load per the HCI principle of chunking.	FR-N03	High	C1
NFR-U03	User Interface	The application shall auto-save in-progress ticket form data to browser memory every 10 seconds to prevent data loss in the event of accidental navigation or session timeout.	FR-N03	Medium	C2
NFR-U04	User Interface	All status labels, button text, and error messages visible to Client users shall use plain, non-technical language. Technical terms (e.g., '500 Internal Server Error') shall be translated to human-readable equivalents (e.g., 'Something went wrong — please try again').	None	Medium	C2
NFR-E01	Evolution	The backend shall be architected using the MVC + Service Layer pattern so that new equipment categories or ticket types can be added by creating new Mongoose models and service modules without modifying existing controllers.	None	High	C1
NFR-E02	Evolution	The AWS ECS Auto Scaling policy shall be configured to maintain 60% average CPU utilization, allowing the system to scale from 2 to 20 task instances automatically within 90 seconds in response to traffic spikes.	NFR-S03	High	C1
NFR-E03	Evolution	Each AI microservice (NLP, Predictive Maintenance) shall expose a /retrain endpoint callable by an Admin to trigger model retraining on updated data without requiring application redeployment.	FR-I05, FR-I06	Medium	C2
NFR-PC01	Constraints	The entire application stack shall be built exclusively using the MERN stack (MongoDB, Express.js, React, Node.js). No SQL databases, PHP backends, or non-JavaScript server runtimes are permitted for core services.	None	High	C1
NFR-PC02	Constraints	The project shall be delivered in 4 phases across 16 weeks, with Phase 1 (Core + Auth) complete by Week 4, Phase 2 (HCI +	None	High	C1

		Portal) by Week 8, Phase 3 (AI) by Week 12, and Phase 4 (Cloud) by Week 16.			
NFR-PC03	Constraints	All credentials, API keys, and database connection strings shall be stored exclusively in AWS Secrets Manager and injected as environment variables at runtime. No secrets shall appear in source code or version control.	None	High	C1
NFR-PC04	Constraints	The project shall use GitHub Actions for CI/CD. Every commit to the main branch must pass linting (ESLint), unit tests (Jest), and a post-deployment smoke test before the ECS service is updated.	None	Medium	C2

5. Requirements Management Tool — RMsis for JIRA

As specified in the assignment, requirements were managed using a formal Requirements Management Tool. **RMsis for JIRA** was selected from the recommended tools list.

TOOL	RMsis for JIRA — Trial account available at: https://marketplace.atlassian.com/apps/30899/rmsis-requirements-management-for-jira

5.1 Why RMsis for JIRA

- Integrates directly with JIRA's issue tracking, enabling requirements to be linked to development tasks and sprints.
- Supports all required attributes: Summary, Category, Dependency, Priority, and Criticality — all of which are captured in the requirements tables in Sections 3 and 4.
- Provides traceability matrices showing which requirements are covered by which test cases, ensuring academic completeness.
- Supports requirement baselining and versioning, which is critical for managing scope changes across the 4-phase roadmap.

5.2 Requirement Attributes Defined

Attribute	Values Used	Definition
Summary	Free text	A concise, implementation-neutral description of what the system must do or be.
Category	Data | Interface | Navigational | Personalization | Transactional | Content | Quality |	The functional or non-functional sub-category the requirement belongs to, as defined in the assignment framework.

	Sys. Env. | UI | Evolution | Constraints	
Dependency	Requirement IDs (e.g., FR-D01) or 'None'	Other requirements that must be implemented before this one can be satisfied. Used for sprint planning and sequencing.
Priority	High | Medium | Low	Urgency of implementation. High = must be in MVP. Medium = required before final release. Low = post-launch enhancement.
Criticality	C1 (Critical) | C2 (Important)	C1 = system fails to meet its core purpose without this requirement. C2 = degrades quality or user experience if missing, but system remains functional.

6. System Architecture & Academic Discipline Mapping

6.1 MERN Stack Architecture

The application adopts a strict three-tier architecture. React.js (SPA via Vite) forms the presentation layer. Node.js + Express.js (MVC + Service Layer pattern) forms the application logic layer. MongoDB Atlas forms the persistence layer. A fourth Python microservice tier (Docker containers on AWS ECS) hosts the AI models.

Layer	Technology	Responsibility
Frontend	React 18, Vite, React Query, Socket.io Client, Recharts, Tailwind CSS	Role-based UI rendering, real-time updates, HCI-optimised workflows
API Server	Node.js 20, Express.js, Mongoose ODM, Socket.io Server, JWT, express-validator	REST + GraphQL API, RBAC middleware, business logic, WebSocket rooms
AI Services	Python 3.11, Prophet (forecasting), Hugging Face sentence-transformers, FastAPI	Predictive maintenance model, NLP ticket categorization and routing
Database	MongoDB Atlas (M10), Mongoose schemas, text indexes, cursor-based pagination	Persistent storage for all 6 core collections with relational references
Cloud / Infra	AWS ECS Fargate, ALB, S3, CloudFront CDN, Secrets Manager, GitHub Actions	Containerized deployment, auto-scaling, CDN delivery, CI/CD pipeline

6.2 Security Architecture

- **Authentication:** JWT with 15-minute access tokens and 7-day refresh tokens stored in HttpOnly cookies (not localStorage) — prevents XSS token theft.
- **Authorization:** RBAC via Express middleware: requireRole('admin'|'technician'|'client') applied per route.
- **Input Sanitization:** express-validator sanitizes all inputs; Mongoose strict mode prevents MongoDB operator injection.
- **HTTPS/TLS 1.3:** Enforced at AWS ALB level with HSTS headers to prevent protocol downgrade attacks.
- **Rate Limiting:** express-rate-limit on auth endpoints (max 10 requests/15 min) to prevent brute-force attacks.
- **Secrets Management:** All API keys and DB credentials stored in AWS Secrets Manager, injected as environment variables at ECS task startup.

6.3 Four-Pillar Academic Coverage Summary

Discipline	Key Features & Implementation	Linked Requirements
Web Engineering	RBAC + JWT security architecture, MVC + Service Layer backend, REST & GraphQL APIs, MongoDB schema design, Socket.io WebSocket real-time updates, server-side pagination and search, HTTPS + input sanitization	FR-D01–D06, FR-I01–I04, FR-N01–N04, FR-T01–T05, NFR-PC01–PC04
Artificial Intelligence	Prophet time-series Predictive Maintenance Engine (nightly cron, Docker microservice). Hugging Face sentence-transformer NLP Ticket Routing (auto-categorization, auto-assignment). Both microservices solve real, expensive business problems.	FR-D05, FR-I05, FR-I06, FR-T03, NFR-Q04, NFR-E03
HCI / UX Design	Formal Heuristic Evaluation (Nielsen's 10 Heuristics). 4-step wizard for ticket creation (recognition over recall, chunking, error prevention). Three role-differentiated dashboards. Plain-language UI labels. WCAG 2.1 AA compliance. Mobile-first Technician PWA.	FR-N03, FR-P01–P03, NFR-U01–U04
Cloud Computing	AWS ECS Fargate containerized deployment (Docker). MongoDB Atlas managed database. AWS S3 pre-signed URL file storage.	NFR-S01–S04, NFR-E01–E03, NFR-PC03–PC04

	CloudFront CDN for React SPA (<50ms TTFB globally). Auto-scaling (2–20 ECS tasks). GitHub Actions CI/CD pipeline.	

7. Implementation Roadmap

Phase	Title	Deliverables & Key Requirements Addressed	Timeline
Phase 1	**Core Infrastructure & Security**	MERN scaffolding, MongoDB Atlas connection, JWT auth, RBAC middleware, User/Client/Equipment models (FR-D01–D06), Equipment Catalog API + UI (FR-N04), Admin CRUD (FR-T04), unit + integration tests (NFR-Q03, NFR-PC04).	**Weeks 1–4**
Phase 2	**HCI-Driven Client Portal**	Persona research & wireframing, heuristic evaluation, 4-step wizard ticket creation (FR-N03, NFR-U01–U04), role-based dashboards (FR-P01–P03), Socket.io real-time updates (FR-I03), responsive mobile-first CSS, PWA for Technicians (NFR-S02).	**Weeks 5–8**
Phase 3	**AI Feature Integration**	Prophet Predictive Maintenance: data pipeline + model training on synthetic dataset (FR-I06, FR-D05). NLP Routing: sentence-transformer microservice + auto-categorization (FR-I05, FR-T01). Admin AI dashboard widgets (FR-T03). Accuracy validation (NFR-Q04).	**Weeks 9–12**
Phase 4	**Cloud Deployment & DevOps**	Dockerize all services (NFR-S03), AWS ECS Fargate + ALB + Auto Scaling (NFR-E02), CloudFront CDN, S3 file storage (FR-I04), GitHub Actions CI/CD (NFR-PC04), MongoDB Atlas M10 (NFR-S04), load testing (NFR-Q01), security audit, WCAG audit (NFR-U01).	**Weeks 13–16**

-------THE END-------